IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 4 September 2025, accepted 9 October 2025, date of publication 17 October 2025, date of current version 28 October 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3622973

SURVEY

Concept Drift Under Harsh Constraints: A Review of Potential Strategies for IoT Systems

ASK ESPENSØNN ØREN[1], NINA KUNA PEUKER[2], AKIRA-MIRANDA ADEYOMI ADENIRAN-LOWE[1], SARAH RUEPP[1], AND MARTIN NORDAL PETERSEN[1]

[1]Department of Electrical and Photonics Engineering, Technical University of Denmark, 2800 Lyngby, Denmark
[2]Department of Applied Mathematics and Computer Science, Technical University of Denmark, 2800 Lyngby, Denmark

Corresponding author: Ask Espensønn Øren (askes@dtu.dk)

This work was supported in part by the Innovation Fund Denmark, through the Research and Innovation Platform Project GreenCOM.

ABSTRACT Concept drift, referring to the temporal evolution of data distributions, presents a significant challenge to the deployment of machine learning models in dynamic, real-world environments. This paper surveys recent advances in autonomous concept drift detection and adaptation methods, with a particular focus on their applicability to resource-constrained platforms such as IoT devices. This survey aspires to be a guideline for choosing a suitable architecture for any situation given the trade-off among detection accuracy, adaptability, and resource efficiency. Our analysis reveals that while statistical methods offer low overhead, they are often insufficient for complex drift scenarios; conversely, more accurate methods may exceed embedded system limitations. We conclude that modular, context-aware approaches that co-optimize both detection and adaptation under tight hardware constraints are essential for practical deployment in embedded learning systems.

INDEX TERMS Concept drift, change detection, TinyML, IoT, incremental learning, drift adaptation, unsupervised learning.

I. INTRODUCTION

Internet of Things (IoT) devices are becoming an increasingly pervasive part of society with implications for several fields including medical, industrial and agricultural. A commonality among these diverse applications is the utilization of machine learning (ML) for extracting actionable insights from the continuous stream of data generated by IoT devices. However, the application of embedded machine learning (EML) in IoT scenarios introduces unique challenges, primarily due to the inherent constraints of IoT devices, such as limited computational resources, memory, and energy supply. Another critical issue in this context is concept drift — a phenomenon where the statistical properties of incoming data streams evolve over time, undermining the predictive performance of machine learning models. Numerous algorithms, such as Adaptive Windowing (ADWIN) [1], Hoeffding's Drift Detection with Moving Weighted Average (HDDMW) [2], and TML-CD [3], have been proposed to address concept drift through detection and/or adaptation mechanisms. However, these approaches often rely on access to a ground truth for effective performance. In industrial and other real-world IoT scenarios, such reliance is impractical, as labels are typically unavailable or require continuous human intervention, a resource-intensive and unscalable approach.

While existing surveys have explored various aspects of concept drift handling, including unsupervised drift detection [4], and even its implications in TinyML environments, there remains a notable gap in examining the feasibility of these frameworks and algorithms from an autonomous IoT perspective, where labeled data is sparse or entirely absent. Several authors have proposed systems capable of identifying drift using scarce or no labels [5], but to our knowledge there exists no newer paper which consolidates the knowledge in the field of unsupervised concept drift handling. This

The associate editor coordinating the review of this manuscript and approving it for publication was Jing Liang.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

review seeks to bridge that gap by critically analyzing the state of the art in concept drift detection and adaptation, focusing on their applicability and efficiency in autonomous IoT settings, thus contributing to the design and deployment of more robust and energy efficient, embedded machine learning systems. Specifically, this paper focuses on on-edge deployment, particularly for MCUs, but extends to discuss systems suitable for light GPUs. In summary the paper aims to:

1) Highlight the recent advancements in (unsupervised) concept drift handling
2) Explore the potential applications for state-of-the art algorithms in the context of resource constrained IoT systems

This paper is structured as follows. Sections II & III provide a formal definition of concept drift and introduces tangentially related fields and how they differ from our scenario. Sections IV and V present state-of-the-art algorithms, as well as key challenges for autonomous drift handling in resource constrained environments for drift detection and adaptation, respectively. Section VI highlights some of the current research directions being taken. Finally, section VII concludes the paper.

II. PROBLEM DEFINITION

Given a data stream $S = \{(\mathbf{X}_1, \mathbf{y}_1), (\mathbf{X}_2, \mathbf{y}_2), \ldots, (\mathbf{X}_t, \mathbf{y}_t), \ldots\}$, where $\mathbf{X}_t$ is the input feature vector and $\mathbf{y}_t$ is the target vector sampled from the joint distribution $\Pr_t(\mathbf{X}_t, \mathbf{y}_t)$, *concept drift* is defined as a change in the underlying data distribution [6]. Using Bayes' Theorem, we can factor the joint probability distribution $\Pr_t(\mathbf{X}_t, \mathbf{y}_t)$ into the conditional distribution $\Pr_t(\mathbf{y}_t|\mathbf{X}_t)$ and the distribution prior $\Pr_t(\mathbf{X}_t)$ as

$$\Pr_t(\mathbf{X}_t, \mathbf{y}_t) = \Pr_t(\mathbf{y}_t|\mathbf{X}_t)\Pr_t(\mathbf{X}_t)$$

A change in $\Pr_t(\mathbf{X}_t, \mathbf{y}_t)$ can be caused by a change in either of the factors. Drift caused by a change in the conditional distribution is known as *real drift*, while drift caused by a change in the prior is known as *virtual drift*. We focus primarily on virtual drift as real drift requires labels to identify [7].

Concept drift is usually divided into four primary types, abrupt, gradual, incremental, and recurring, characterized by the rate of change, as well as whether the change has a form of periodicity. Drift detection algorithms are typically specialized for a small subset of drifts, as their characteristics make it difficult to design an optimal one-size-fits-all algorithm, although some methods, like Meta-ADD [8], attempt to address this issue by identifying drift type in parallel to drift occurrence.

For example, abrupt drift is characterized by a sudden, significant, change and is easily noticeable when observing a fairly small window of incoming data. Incremental drift, on the other hand, manifests itself as a smooth change over a longer period of time, typically requiring a prior (stationary) reference to detect. In gradual drift, the concept is changing slowly over time, with the rate of change suddenly increasing. Lastly, in recurring drift the same concepts appear repeatedly, sometimes periodically such as day/night cycles. Figure 1 expands on the drift representation in [7] to show the behavior of the respective drift types. It should be noted that recurring drift is sometimes omitted as it could be defined as a series of other drift types. Consequently, a system capable of continuously adapting to other drifts will by definition also be capable of handling recurring drift. However, it is still included here as if recurring drift is known to be common for the system, one could feasibly design systems to target recurring drift, thus reducing adaptation latency.

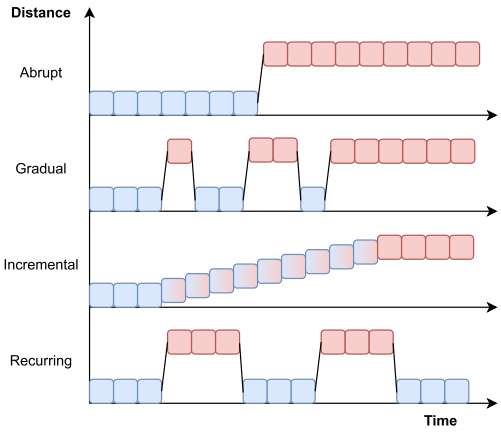


FIGURE 1. The four primary drift types.

III. RELATED FIELDS

A. CONTINUAL LEARNING

In machine learning, continual learning, also known as lifelong learning, relates to the problem of training machine learning models in a non-stationary setting. Consequently, it is in many cases similar to concept drift adaptation, except that it also aims to expand the capacity of the networks to learn new tasks, as opposed to just adapting to a change in an existing one [9].

A core research focus in continual learning is overcoming *catastrophic forgetting*, that is, forgetting old tasks when learning new ones. Several systems have been suggested to this effect, such as regularization methods [10], selective retraining [11], and complementary memory systems [12]. However, the issue remains an active research area.

Aside from systems targetting recurring drift, catastrophic forgetting is generally not a concern when targeting concept drift. As long as the adapted system retains accuracy post-drift it is inconsequential whether the same model would be accurate under the previous concept. Moreover, the added memory-overhead associated with the prerequisite memory retention mechanisms mean these methods are typically not directly implementable onto IoT systems.

B. UNSUPERVISED DOMAIN ADAPTATION

Unsupervised domain adaptation (UDA) studies the problem of distribution shifts when an unlabelled data set from the

test distribution is available at training time, in addition to a labelled data set from the training distribution [13]. While sharing similarities, UDA is therefore fundamentally different from concept drift adaptation in the challenges they address. Concept drift addresses temporal shifts in the data distribution of unknown type and severity happening at (a) random point(s) in time, while in UDA the target distribution is assumed to be stationary and accessible during training.

C. ANOMALY DETECTION

The problem of handling concept drift is closely intertwined with the detection of anomalous points and sequences and the detection of out-of-distribution (OOD) samples. Anomalous sequences denote a continuous anomalous pattern in the data stream over a continuous time frame [14], [15]. Thus anomaly detection could be considered as a field related to concept drift, especially gradual and recurrent drift. Current state-of-the-art anomaly detection methods in machine learning applications most often rely on long-short-term-memory (LSTM) models and variational auto-encoders [16], [17], an approach that is explored in concept drift detection as well [18]. However, precisely because anomaly detection and concept drift detection attempt to solve similar problems, the presence of one (i.e. concept drift) in a system severely complicates the detection of the other (anomaly), leading to an increase to false alarms or anomalies being wrongfully classified as concept drift. Some drift detection methods such as VAE4AS [14] therefore specialise in drift detection in the presence of anomalous sequences, and vice versa.

IV. CONCEPT DRIFT DETECTION

Concept drift handling is typically divided into two parts; detection and adaptation. Over the past decades, much work has been put into developing and optimizing drift detection methods, summarized for example by [19], [20] and [21]. The most common drift detection strategy uses an error metric to detect drift with common examples including Drift Detection Dethod (DDM) [22], Early Drift Detection Method (EDDM) [23] or the aforementioned Meta-ADD [8]. However, error measurements require access to the true labels of incoming data instances as opposed to unsupervised methods which work without any information about true class labels. Some methods, such as ADWIN [1], Page-Hinkley (PH) [24] and CUSUM [25], present monitoring techniques that can be applied just as well to statistical characteristics of the data stream as they can to error rates. Common choices for these statistical characteristics include mean and variance. Depending on the model setup, these methods can be classified either as supervised or unsupervised detection methods.

When further distinguishing detection methods from each other, we orientate ourselves on the taxonomy of unsupervised detection methods introduced by Gemaque et al. [4], distinguishing between online- and batch methods. According to them, detection methods can be differentiated based on whether they check the incoming data for drift instance-by-instance (online) or collect them in batches first, then decide on drift occurrence for an entire batch at a time (batch). For batch methods it is important to know whether they make use of all instances within the batch or only consider some, while online methods can be distinguished by whether the reference window is fixed or sliding. The choice between a fixed and sliding reference window as well as the window size can impact the sensitivity of the method towards certain types of drift. Having a fixed reference window provides historic context needed to reliably identify recurrent drift, while the adaptability of sliding reference windows support the accurate detection of gradual or incremental drifts. Abrupt drifts are relatively easy to detect using both fixed and sliding windows, though the window size has to be adjusted accordingly. Reference [26] put it quite succinctly: ''Larger windows correlate with higher performance accuracy, however, they may also contain concept drift within themselves that could escape unnoticed. Smaller windows tend to facilitate better concept drift detection''. On the other hand, if windows are too small, certain drift types, like incremental and gradual, are easily missed. The choice of window size therefore represents a trade-off between false positives and false negatives.

In the following, we will investigate a selection of unsupervised drift detection methods. The purpose of this paper is to determine the feasibility of state-of-the-art unsupervised drift detection methods for IoT devices, therefore, the primary focus of this investigation lies on 4 main aspects:

1) the computational requirements necessary to detect drift
2) the detection latency of the method
3) the involvement of an auxiliary model
4) the types of data that can be monitored

Subsequently, we focus on potential limitations for each method, especially when applied in IoT scenarios.

A. STREAM METHODS

Online learning is a term for models that are continuously updated when new data arrives. Stream-based methods represent a subset of online learning in which the incoming data stream one is processed instance at a time. When formally defining the incoming data stream $S = \{B^t\}_{t=1}^{T}$ as a collection of incoming batches $B^t = \{(x_i^t, y_i^t)\}_{i=1}^{N}$, where N is the batch size [27], stream based detection methods process S in batches of size $N = 1$, as opposed to batch learning where $N > 1$.

The cumulative sum algorithm (CUSUM) [25] and its variant the Page-Hinkley test (PH) [24] are some of the oldest and most widely adapted strategies to detect concept drift and shifts in data. First published in the 1950s, these methods make use of monitoring the cumulative difference from a certain metric, often sample mean or variance, processing incoming data one sample at a time. CUSUM assumes a ground truth or initial value, whereas

PH continuously updates the current metric, making it more adaptive. If the cumulative sum exceeds a certain threshold, a drift is detected. This setup makes PH/CUSUM extremely lightweight, adaptive, and versatile all in constant time complexity. However, its simplicity also limits the types of drift it can detect. Due to its adaptivity, PH is more robust against noisy data, which comes at the cost of its latency, making it mainly suited for abrupt drift, while CUSUM is more sensitive to noisy data, but handles gradual drifts as well.

Another example of a simple drift detection method is the Incremental Kolmogorov-Smirnov test (IKS) introduced by dos Reis et al. [28]. IKS builds on the well-established Kolmogorov-Smirnov test [29], a distribution independent goodness-of-fit test which determines whether two samples are likely drawn from the same, but achieves a linear time decrease through random trees. According to the test, the null hypothesis of two samples A and B originating from the same distribution, can be rejected at level α, when the following inequality is satisfied:

$$\sup_x |F_A(x) - F_B(x)| > c(\alpha)\sqrt{\frac{n+m}{nm}} \tag{1}$$

where n and m denote the number of observations in A and B respectively. The Kolmogorov-Smirnov test can be a reasonable choice for a change detector due to its computational efficiency, but does not scale well with infinite data streams, as the information retained from previous samples grows indefinitely. In terms of equation 1, this means that A and B accumulate samples from the stream indefinitely, making the supremum operation increasingly expensive. The incremental version of this test uses random trees to mitigate this problem. The tree structure allows for efficient recomputing of the KS statistic when inserting or removing of samples. Given a fixed reference window and a sliding current window, IKS can perform the statistical test for each new sample in $O(1)$ time, while the window movement and tree update takes $O(\log N)$ (N denoting the number of observation per window) time, compared to the $O(N \log N)$ required by the original test.

For certain types of machine learning algorithms, the model structure itself can be used to detect concept drift. Simple classifiers, for instance, allow drift detection through monitoring the confidence in their prediction. Examples for such classifier confidence methods are the Confidence Distribution Batch Detection (CDBD) [30] or the Margin Density Drift Detection (MD3) [31]. Both detect drift by tracking the uncertainty of a classifier such as a Support Vector Machines (SVM) [32]. While CDBD tracks the actual confidence distributions for each prediction, MD3 focuses on the change in uncertainty margins. This is done either by monitoring the margin densities (for SVM), or the computed Blindspot Density (BD) [31], shown in equation 2.

$$\mathrm{BD}_{RS} = \frac{\sum S_E(x)}{|X|};\ \forall x \in X \tag{2}$$

where S_E denotes the signal function, which checks whether a sample x has a certainty of less than the *margin of uncertainty* θ:

$$S_E(x) = \begin{cases} 1 & |\Pr_E(y=-1|x) - \Pr_E(y=1|x)| \leq \theta \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

To address challenges associated with high dimensional feature spaces, [33] introduces ODIN, a visual data analytics system which automatically detects and recovers from drift. The detector in ODIN uses a Dual-Adversial Generative Adversial Network (DA-GAN) to learn a low-dimensional representation of the data, which is then used to cluster the data points. From the clusters, density bands are computed to create a representation which only captures the high-density regions of the clusters. These density bands are defined by the upper and lower bounds which with $\Delta = 0.5$ hold at least half the observations in the cluster. The detector computes these bounds using the density function f:

$$\int_{\Delta_l}^{\Delta_h} f_\Delta(x)\,dx\ = \Delta \tag{4}$$

Drifts are detected by comparing the distribution of new incoming data points to the existing clusters using the Kullback-Leibler divergence [34]. Any incoming point is first projected into the low-dimensional manifold using the DA-GAN. If the point then falls into one of the existing clusters' density bands, it is assigned to that cluster, otherwise it is added to a temporary cluster. Once a temporary cluster stabilises, a drift is detected. Signalling a drift whenever the temporary cluster stabilises provides robustness, but in turn leads to a higher latency than other detection methods as the detector alerts once the drift is completed, not when it starts.

As discussed in earlier sections, supervised methods tend to be more accurate and reliable in detecting drifts, but are inconvenient due to the lack of true labels or performance indicators. With the Student Teacher approach for Unsupervised Drift Detection (STUDD) [35], Cerqueira1 et al propose a way to approximate the error of a model in an unsupervised scenario by employing a knowledge distillation strategy called "Student-Teacher". The knowledge learned by a larger "teacher" model is distilled into a smaller "student" model by training the student on the teacher's predictions. In a concept drift scenario, the authors hypothesize that a drift reveals itself in the discrepancy between the predictions of the teacher and the learner. To detect this drift, the PH detection method is applied, but not to the error rate of the teacher model or any metrics in the incoming data stream, but to the difference in predictions between student and teacher. The STUDD method thus aims to replace true labels with mimicked labels recovered from the student model. Its behaviour is "conservative" [35] compared to other supervised state of the art models, making it suited for environments in which false alarms and true label acquisition for retraining are expensive. The drawback of this approach, especially in low-resource and low-memory environments, is the additional storage required for a second model.

B. BATCH METHODS

Contrary to stream methods, batch methods process incoming data instances in batched B_t of size $N > 1$. In some cases, N is an adaptive parameter and changes depending on the drift behaviour [1].

Despite being introduced in 2007, ADWIN [1] remains one of the most versatile and prominent detection methods. Using a sliding window W of adaptive size, ADWIN monitors an indicator, such as error-rate in the supervised case, classifier confidence, or any metric from the incoming samples in the unsupervised case. Instead of being determined by the user, the window size $n = |W|$ is adjusted according to the change observed in the data. Using a statistical test, ADWIN decides whether there is a significant difference between any two sub windows in W. If that isn't the case, the window size $n = |W|$ is extended, whereas if a change is detected, the window size is decreased. This makes ADWIN more flexible and tailored to the incoming data, increasing performance compared to similar methods like [22]. One of the main achievements of ADWIN is the presentation of upper bounds on false positive and false negative rates. For an input parameter $\delta \in (0, 1)$, ADWIN caps the false positive rate at δ and guarantees a true detection alert with probability δ if the difference in observed mean between two subwindows W_0 and W_1 of sizes n_0 and n_1 exceeds $2\epsilon_{cut}$, where

$$\epsilon_{cut} = \sqrt{\frac{1}{2m} \ln(\frac{4n}{\delta})} \qquad m = \frac{1}{1/n_0 + 1/n_1} \qquad n = n_0 + n_1 \tag{5}$$

However, in its initial form the method is quite computationally expensive, as it needs to check any possible number of sub windows in W for each example. A solution is presented in ADWIN2 [1], which specifies a number of cut points c through an input parameter M, where $c = 1 + 1/M$. This small adjustment takes the time complexity to $O(\log^2 W)$ for each sample and $O(\log W)$ of space required.

In order to capture more complex drift phenomena, some works instead rely on space partitions and density estimations. The Hellinger Distance based Drift Detection Method (HDDDM) [36] constructs histograms with $\sqrt{N}$ bins for each feature and window and monitors the difference to the (fixed) reference histogram using the Hellinger Distance:

$$\sqrt{\sum_{i=1}^{\lfloor\sqrt{N}\rfloor} (\sqrt{P_{i,k}} + \sqrt{Q_{i,k}})^2} \tag{6}$$

Using an adaptive threshold, the difference in Hellinger distance between the distribution in the sliding window and the reference distribution indicates whether or not a drift has occurred. While HDDDM is fairly efficient both in terms of computational efforts and memory requirements, both scale with the dimensions of the feature space.

Similarly to HDDDM, NN-DVI [37] relies on density estimations of incoming features. NN-DVI consists of 3 distinct steps: first, the authors introduce a novel neighbour-based partition schema (NNPS) to create a partition from the incoming data. The NNPS based dissimilarity measure relies on multiset theory to accumulate the difference between instance particles.

$$d_{\text{NNPS}}(M_A, M_B) = \frac{1}{|P(A) \cup P(B)|} \cdot \sum_{p_{d_i} \in P(A) \cup P(B)} \frac{|I_{M_A}(p_{d_i}) - I_{M_B}(p_{d_i})}{I_{M_A}(p_{d_i}) + I_{M_B}(p_{d_i})} \tag{7}$$

Here M_A, M_B denote two instance particle sets, $I : X \to \mathbb{N}$ the multiset indicator function and p_{d_i} an instance particle combining an instance d_i with information about its neighbours.

Using the constructed NNPS and leveraging the distance measurement above, the authors then introduce a tailored statistical test to detect variation in the partition schemes. NN-DVI performs well for small regional drifts and preserves its power even for high-dimensional data [37]. On the flipside, the distance and NNPS computations performed for each patch add significant computational strain on any host device. Latency or detection time, in RAM hours, is comparable to other detection methods such as ADWIN, HDDM or KL [37].

Moving away from statistical insights into the incoming data stream, the Discriminative Drift Detector (D3) [38] employs a separate model for drift detection. D3 bypasses the issue of relying on information from either the distribution of the data stream or the model structure by applying a sliding window and training a simple classifier, for instance a logistic regression classifier, on distinguishing samples within and outside the sliding window. If the classifier performs well, there is reason to believe that samples in the more recent subwindow are significantly different from previous samples. On the other hand, a poor classifier performance indicates no significant, learnable differences between previous and current samples, suggesting that both streams originate from the same distribution. The classifier performance is measured using its AUC score. This approach presents an opportunity to detect drift without monitoring the distribution or the model performance directly and works entirely without data labels.

The growing dimensionality of feature spaces makes unsupervised concept drift detection progressively more challenging. Luckily, the increase in complexity of contemporary machine learning models presents more and more opportunities to detect the impact of the incoming data stream on the model in ways that are not directly tied to performance. Drift Lens [7] explores the concept of embedding spaces in deep learning architecture and detects drift by monitoring the distribution distances in the embedded structures of the model. The embeddings are reduced in their dimensions using PCA, modelled as multivariate normal distributions and their distance to a reference distribution is monitored using the Frechét distance:

$$\text{FDD}(b, w) = ||\mu_b - \mu_w||_2^2 \cdot \text{tr}(\Sigma_b + \Sigma_w - 2\sqrt{\Sigma_b \Sigma_w}) \tag{8}$$

where both baseline distribution b and the distribution in the current window w are described by their mean vectors μ_b, μ_w and covariance matrices Σ_w, Σ_b. This makes Drift Lens an effective method to detect drift in unstructured data, since it focuses less on the raw data and rather on the representations made within the model. Additionally, Drift Lens provides drift information both per-batch and per-label, making the handling more focused and the drift easier to mitigate.

Lastly, PCA has also been explored in the context of concept drift detection. Kuncheva et al. [39] showed that performing a PCA based feature extraction before applying unsupervised drift detection methods can improve the overall detection performance. Later, Qahtan et al. [40] proposed a PCA-based change point detection method and showed that changes in correlation, mean and variance could be detected using PCA. Finally, Wahab [41] presented yet another PCA-based detection method (PCA-DD), specifically for IoT devices and intrusion detection. The method utilises the assumption that a change in feature distribution can be detected by investigating the principal components that span the feature space. For each window, the feature covariance matrix and its eigenvectors are computed; a drift is detected whenever the angle between the eigenvectors exceeds a certain threshold. The PCA-DD is developed specifically for IoT scenarios, although depending on the environment constraints, the additional computational effort necessary to compute both the covariance matrix and its eigenvalues for every batch may exceed the tight resources available on IoT devices. The computational effort increases with the dimensions of the feature space.

C. COMPARISON

The presented methods are summarised together with any consideration for their applicability in IoT in table 1.

D. DISCUSSION

Based on the findings summarised in Table 1, we now assess the feasibility of implementing the examined detection methods on IoT or other resource-constrained platforms.

Statistical detectors such as ADWIN and PH, which rely on monitoring basic summary statistics, exhibit minimal memory requirements and computational overhead. However, their detection capabilities are largely restricted to simple forms of drift, such as changes in mean or variance, in mainly univariate data streams. Consequently, these methods may be insufficient for more complex scenarios.

Methods like HDDDM and IKS, while requiring slightly more memory than ADWIN and PH, offer the ability to detect more intricate types of drift. Their time and space complexities remain linear in window size and feature dimensionality, rendering them sufficiently tunable for embedded contexts. A key limitation, however, lies in their univariate treatment of features, which risks missing drifts arising from inter-feature interactions—an important consideration when dealing with multivariate data.

Methods such as MD3 are extremely efficient in both computational and memory requirements and useful for reducing false alarms, which present big issues in previous methods. That said, their application is limited to a small set of models, consisting of robust classifiers that provide notions of margins or classifiers for which a BD can be computed. This limits the scenarios in which MD3 can be applied to severely.

NN-DVI presents a specialised approach tailored towards small, localized drifts and is theoretically appealing due to its manageable computational complexity. Nonetheless, this theoretical efficiency does not appear to materialise in practice. For instance, Lukats et al. [42] reported that a straightforward implementation of NN-DVI, following the authors' instructions [37], proved ''too slow to allow thorough evaluation'' [42]. While this limitation may be partially attributable to the use of scikit-learn, the feasibility of NN-DVI in constrained environments remains doubtful. Broadly speaking, unless further optimisation strategies are incorporated, methods relying on repeated nearest-neighbour operations appear ill-suited for resource-constrained deployment.

Model-assisted detection frameworks, such as ODIN (DA-GAN), STUDD (Student), and D3 (discriminative classifier), demonstrate strong performance across diverse data streams. In fact, STUDD and D3 emerge as top-performing methods in several recent benchmarking efforts [42], [43], while ODIN has shown particular success in reducing the overhead associated with high-dimensional input data [33]. While these methods offer strong empirical performance with tolerable runtime overhead, the additional memory cost of maintaining an auxiliary model must be carefully considered for IoT use cases.

Lastly, methods that incorporate dimension reduction via PCA, such as Drift Lens [7], exhibit feasibility that is highly dependent on how PCA is utilised. In scenarios where incoming data is projected onto precomputed principal components, computational demands remain moderate. However, applying singular value decomposition (SVD) to high-dimensional embeddings for every incoming batch quickly becomes intractable. Despite claims by the authors of Drift Lens [7] regarding its low complexity and real-time capability, doubts remain about its applicability in highly constrained environments.

V. CONCEPT DRIFT ADAPTATION

Concept drift adaptation is typically split into three main types: active methods, passive methods, and hybrid methods. In active methods, also referred to as informed adaptation methods [6], the model is only updated when a drift is detected [27].

Passive methods, on the other hand, gradually update the model, either at each incoming data sample, after a batch of samples, or after a certain time period [27]. In order to update, passive methods typically rely on a sliding window, like FLORA [44] and CVFDT [45], or more recently, ensemble

TABLE 1. Summary of concept drift detection methods surveyed in this paper. Computational and memory requirements are classified in terms of the dimensions of the feature space and window sizes as: Very Low - sublinear; Low - linear and logarithmic; Moderate - linear to polynomial with degree ≥ 2; High - polynomial with degree > 2; Very High - exponential. *naive implementation has proven inefficient.

Method name	Category	Computational requirements	Detection latency	Auxiliary model	Types of data
ADWIN [1]	batch sliding window	very low in time very low in space[1]	low	no	univariate and simple multivariate
PH/CUSUM [24,25]	stream sliding window (PH), fixed window (CUSUM)	very low in time very low in space	very low	no	univariate
IKS [28]	stream fixed window	low in time low in space	low	no	univariate and independent multivariate
Drift Lens [7]	batch fixed window	high in time moderate to high in space	low	no	high dimensional
D3 [38]	batch sliding window	low in time low in space	low to moderate	yes	any
HDDDM [38]	batch fixed, incremental window	very low in time very low in space	low	no	univariate and independent multivariate
NN-DVI [37]	batch fixed window	low to moderate in time very low in space*	moderate	no	any
STUDD [35]	stream fixed window	very low in time very low in space	very low	yes	any
PCA-DD [41]	batch sliding window	high in time moderate to high in space	low	no	multivariate
ODIN [33]	stream fixed, incremental window	moderate to high in time moderate to high in space	moderate to high	yes	high dimensional
MD3 [31]	stream fixed window	very low in time very low in space	very low	no	univariate

methods, such as H3E [46], which utilize multiple models that can be dynamically added, removed, and weighted against each other based on their performance.

Hybrid methods, such as HAREBA [47], combine the two by updating both upon detection and performing gradual updates.

Aside from the aforementioned three, recent works have also explored drift unlearning, i.e., reversing the changes to the distribution, thereby foregoing the need to update the downstream models. You and Lin [48] proposed augmenting traditional adaptation algorithms, adding an additional "unlearning test" to check whether forgetting older data points would improve accuracy by comparing the loss of the model with and without certain data points. Later, Artelt et al. [49] follow the same underlying principle, but does so using an autoencoder to learn the inverse mapping function from the new concept to the old in an unsupervised manner.

Whereas drift detection is typically not a resource-intensive task, adaptation often involves re-training neural networks [21]. Consequently, in resource-constrained systems, such as IoT devices, effective adaptation algorithms necessitate balancing effectiveness with complexity. While this has yet to be resolved, several methodologies have been proposed, especially in recent years. In the following section, we delve into a subset of them, focusing on those that minimize human intervention.

A. ONLINE LEARNING

Several studies of concept drift adaptation for tiny machine learning have employed online learning because of its ability to constantly integrate new information without storing historical data. For example, TinyOL [50] presents an online supervised learning algorithm that adds a small trainable network on top of, or replaces, the final layer of a pre-trained base model. The extra network allows otherwise static models to be trained post-deployment, essentially performing online transfer learning. Using this approach, the authors were able to produce an almost identical distribution in the mean-squared error both before and after the occurrence of concept drift.

As labels are typically absent or scarce when working with IoT streaming data, several alternatives to supervised paradigms have also been developed. VAE4AS [14] presents an unsupervised paradigm for learning out-of-distribution stream detection. They adopt a two-pronged approach, using two detection heads to validate when a drift has occurred, and a variational autoencoder (VAE) for adaptation.

In another approach, TML-CD [3] uses a k-NN classifier with a fixed pre-trained feature extractor. Adaptation is achieved by modifying the samples in the knowledge base of the k-NN. This method is a form of incremental clustering, in which out-of-distribution samples can be found by comparing them with the existing knowledge base.

Digital twins, i.e., a simulated clone of the physical system, have also been used in combination with online learning for concept drift adaptation. TWIN-ADAPT [51] is an example of such systems for clean rooms. It employs a dual-window strategy for monitoring and adaptation, using LightGBM [52] as a base model. However, one inherent issue with using digital twins for IoT systems is that they are usually too complex to model effectively.

B. NEURAL ARCHITECTURAL SEARCH (NAS)

Neural Architecture Search (NAS) automates the creation of neural networks by employing an algorithmic approach to explore candidate network structures [53]. NAS is particularly relevant in the context of tinyML as it allows resource constraints to be directly incorporated into the architecture design process. This enables the automatic discovery of architectures that are not only resource efficient, but also meet the latency requirements of tiny devices.

HardCoRe-NAS [54] targets latency-constrained systems, and introduces a differentiable approach that enforces hard resource constraints during the search process, rather than treating them as soft penalties. The search is formed as a hard-constrained optimization problem which accounts for both micro-level block configurations and macro-level network depth configurations. It trains a single one-shot model using multi-path sampling, which is a technique that evaluates multiple sub-networks per training batch. Specifically, the NAS is expressed as the following bilevel optimization problem:

$$\begin{aligned} &\min_{\zeta \in \mathcal{S}} \ \mathbb{E}_{x,y\sim\mathcal{D}_{\text{val}}}\left[\mathcal{L}_{\text{CE}}\left(x, y \mid w^*, \hat{\zeta}\right)\right] \\ &\hat{\zeta} \sim \mathcal{P}_\zeta(\mathcal{S}) \\ &\text{s.t. } \text{LAT}(\zeta) \leq T \\ &\quad w^* = \arg\min_{w} \ \mathbb{E}_{x,y\sim\mathcal{D}_{\text{train}}}\left[\mathcal{L}_{\text{CE}}(x, y \mid w, \hat{\zeta})\right] \\ &\quad \hat{\zeta} \sim \mathcal{P}_\zeta(\mathcal{S}) \end{aligned} \tag{9}$$

where $\mathcal{D}_{train}$ and $\mathcal{D}_{val}$ are the training and validation sets, respectively, $\mathcal{P}_\zeta$ is a probabilistic measure over the fully-differentiable search space parametrized by $\zeta = (\boldsymbol{\alpha}, \boldsymbol{\beta})$. Finally, $\mathcal{L}_{CE}$ is the cross-entropy loss, and LAT(ζ) is the accuracy of the model. The authors solve equation 9 using Block Coordinate Stochastic Frank-Wolfe optimization [55], a variant of the Franke-Wolfe update rule that ensures strict constraint satisfaction during the search process.

Similarly to HardCore-NAS, E-DNAS [56] employs a differentiable neural architecture search approach, specifically designed for embedded systems. It uses depth aware convolution to capture high-resolution feature maps, followed by parallel architecture search that learns the optimal sizes and parameters of meta-kernels. E-DNAS uses the additive property of convolution operations to merge multiple kernels with compatible sizes into single operations. As additivity is also applicable for batch normalization, operations can be further condensed as:

$$\boldsymbol{O} = \boldsymbol{I} \cdot \left(\frac{\gamma_1}{\sigma_1}\boldsymbol{K_{3x3}} \oplus \frac{\gamma_2}{\sigma_2}\boldsymbol{K_{3x1}} \oplus \frac{\gamma_3}{\sigma_3}\boldsymbol{K_{1x3}}\right) + b \tag{10}$$

s Where $\boldsymbol{O}$ is the output feature map, $\boldsymbol{I}$ is the input data, $\boldsymbol{K}$ are filters of compatible sizes, $\oplus$ is the element-wise addition operator, σ is the batch standard deviation, and γ and b are the learnable batch normalization parameters. To obtain the meta-kernels E-DNAS uses a feedback-block mechanism that updates learned meta-kernels during training through weighted sums based on the loss values from parallel training paths, i.e.:

$$\boldsymbol{K}_1' = \boldsymbol{K}_2' = \beta_1 \cdot \boldsymbol{K}_1 + \beta_2 \cdot \boldsymbol{K}_2 \tag{11}$$

where β_i are computed weights defined such that they will be close to one for small losses. In order to attend to both classification accuracy and latency, a multi-objective loss function is used. However, noting that the correlation between MACs and latency is device dependent and often non-linear [57], [58], E-DNAS instead employs a look-up table with sampled latency values from the target platform.

μNAS [59] is a constrained NAS method specifically designed for finding optimal neural-architectures for microcontrollers. The method addresses MCU constraints by constructing architectures as Directed Acyclic Graphs (DAGs) with precise control over layer parameters. These architectures are then evaluated against three resource modelling components; peak memory, network size and latency. μNAS accounts for execution order dependencies in branched networks, as different operation sequences require varying peak memory, by enumerating across all valid execution orders to determine the minimum memory requirement. Model size estimation is done through parameter counting with 8-bit quantization, while latency estimation uses multiply-accumulate operations (MACs) as a proxy for its true value. The search process combines multi-objective optimization using ageing evolution [60], whilst randomly weighting the importance across the accuracy, memory, storage, and latency objectives. The method integrates structured pruning during search, training larger base architectures before removing components rather than training small networks directly. Contrary to HardCore-NAS, however, μNAS employs soft-constraints, which should be taken into account when setting the constraints to ensure the found architecture can be used on your given device.

While both μNAS [59] and MSNAS [61] employ a graph-based search strategy using DAGs, their approaches differ significantly. MSNAS introduces a probalistic meta-graph that represents the entire search space and accumulates learned knowledge throughout the process. In this framework, candidate architectures are DAGs sampled from the meta-graph. Each node is represented by a node operation and produces an output tensor. The edges $e_{u\to v} = (u, v, w_{\text{uv}}) \in \mathcal{E}$ represent the flow of tensors, with the weight, w_{uv} being the probability of two nodes being connected. Search is done through a sequential

process of DAG sampling, multi-objective evaluation of accuracy and hardware constraints, and selective weight updates in the meta-graph. On the notion of objective evaluation, MSNAS adopts a structural wiring architecture that reduces dependency between nodes, allowing for more compact models without sacrificing accuracy. The search metric combines performance metrics such as accuracy with IoT-specific metrics like MACs and peak SRAM usage to find a wide range of neural architectures capable of fitting various resource constraints:

$$\mathcal{M}(\mathcal{A}) = \text{Perf}(w^*(\mathcal{A}), \mathcal{A}, \mathcal{D}) - \text{IoT}(\mathcal{A}) \tag{12}$$

$$w^*(\mathcal{A}) = \arg\min_w \mathcal{L}_{\text{train}}(w, \mathcal{A}) \tag{13}$$

$$\text{IoT}(\mathcal{A}) = \lambda_a \left\| \sum_i^n k_i \cdot \text{adj}(\mathcal{A}) \right\|_0 + \lambda_b \cdot \text{MAC} + \lambda_c \cdot \text{param} \tag{14}$$

where *Perf* () denotes the performance metrics of the current candidate architecture given optimal weights on a validation set. Additionally, MSNAS proposes post-search adaptation through structure-level pruning of the meta graph that removes connections below specified thresholds, further alleviating hardware constraints.

Co-Design NAS [62] employs a "co-exploration" framework that simultaneously optimizes neural architecture, quantization scheme, and hardware implementation, exemplified for FPGA platforms. For the architecture and quantization spaces the authors employ reinforcement learning with a LSTM controller, which is updated using the Monte Carlo policy gradient algorithm, while a multi-objective search algorithm optimizes hardware space exploration. Co-optimization of bit-widths for weights and activations, as well as architectural parameters, is achieved through linear quantization with fixed-point representation. While theoretically applicable for a plethora of hardware, the work is exemplified for FPGAs, using tile-based paradigms to account for both intra-layer parallelism and inter-layer mapping to FPGAs, with dynamic programming used to search the hardware space. The approach validates each sampled network against design specifications before training. While the co-design framework achieves better flexibility and offers more compression, the authors also find that the addition of early-stage hardware search leads to more variance and early convergence to (worse) local optimality. Moreover, using reinforcement learning means that any small parameter adjustment requires full retraining of the network, which introduces severe costs to training time.

HIO-NAS [63] proposes a hardware-aware iterative one-shot approach that sequentially searches the relevant spaces. One-shot NAS like [64] train a singular supernet consisting of intertwined sub-architectures which are sampled during inference. Due to weight-sharing the total computational costs are significantly reduced compared to, for example, reinforcement learning based NAS. HIO-NAS decomposes the multi-objective NAS problem by iteratively searching depth, filter, and width spaces sequentially. It is comprised of six key steps; defining a supernet, selecting a search space, performing a full train and search in a one-shot fashion, retraining the most promising candidates, and finally updating the supernet based on best-performing architecture. Prior to supernet training, HIO-NAS uses pre-constructed lookup tables to evaluate hardware constraints including latency, FLOPS, and parameter counts. This way, only subnetworks satisfying the hardware requirements are evaluated during model inference. Finally, it applies adaptable knowledge distillation with dynamic teacher weighting during re-training. Essentially, the best candidate sub-network from the supernet is designated as the student network. To guide it, another sub-network is randomly selected from the same supernet to serve as the teacher model. Yet, as using the teacher predictions directly in the loss function is unsuitable, it takes inspiration from [65] and makes use of a temperature:

$$\hat{q}_i = \frac{\exp(\frac{q_i}{T})}{\sum_j \exp(\frac{q_j}{T})} \tag{15}$$

The system is adaptive as the teacher is only used for guidance when the loss of the student starts to stagnate, or when the performance reaches a pre-defined threshold.

C. TEST-TIME ADAPTATION

Test-time adaptation (TTA) [66] refers to methods designed to modify a model's behaviour during inference in order to adapt to distribution shifts encountered in incoming data. TTA methods are typically designed to be passive methods, without a detection step deciding when to perform an update. Unlike traditional passive approaches, which rely on periodic retraining or fixed adaptation schedules, TTA methods perform model updates at the moment of prediction, enabling immediate response to environmental changes without requiring labelled data from the target distribution.

Batch-normalization methods, like the one proposed by Schneider et al. [67], propose a paradigm of TTA which automatically account for shifts occurring in the covariance matrix by recomputing normalization statistics from every test batch as a weighted combination of training (μ_s, σ_s^2) and test statistics (μ_t, σ_t^2), formulated as:

$$\bar{\mu} = \frac{N}{N+n}\mu_s + \frac{n}{N+n}\mu_t \tag{16}$$

$$\bar{\sigma}^2 = \frac{N}{N+n}\sigma_s^2 + \frac{n}{N+n}\sigma_t^2 \tag{17}$$

The parameter N represents the trade-off between the source and target statistics where as $N \to \infty$, only training statistics are used.

Similarly, Zachary et al. [68] expand upon this idea but focus on the incoming batch, foregoing the need for frozen training statistics. This is in contrast to the typical strategy of normalizing neural network activations using statistics computed during training, where the mean $\mu^{(c)}$ and variance $\sigma^{(c)^2}$ are calculated from training batches and then frozen

as exponential moving averages for use during inference. Instead, $\mu^{(c)}$ and $\sigma^{(c)^2}$ are now recalculated for each incoming test batch to account for covariate shift.

Leveraging the principle that confident predictions, i.e., low entropy, have a strong correlation to high accuracy, TENT [69] introduces test-time entropy minimization as a fully test-time adaptation method that requires only the model parameters and unlabelled target data. It adapts pre-trained models during inference by minimizing the entropy of the predictions on test data. TENT updates only the normalization layers of the network during adaptation. Specifically, it re-estimates the batch normalization statistics from target data and optimizes the affine parameters for scale γ and shift β through gradient descent on the entropy loss (equation 18) for the probability $\hat{y}_c$ of the class c. These parameters are selected for updating to prevent the model from diverging too far from the source distribution.

$$H(\hat{y}) = -\sum_{c} \Pr(\hat{y}_c) \log \Pr(\hat{y}_c) \tag{18}$$

Marginal Entropy Minimization with One test point (MEMO) [70] extends the entropy minimization principle by focusing on single test samples and augmentation-based adaptation. Unlike TENT, which requires batches of test data, MEMO works with individual test points by generating multiple augmented versions and minimizing the marginal entropy across them. The method adapts all model parameters rather than just normalization layers, using the objective:

$$\ell(\theta; x) = H(\bar{\Pr}_\theta(\cdot|x)) = -\sum_{y \in Y} \bar{\Pr}_\theta(y|x) \log \bar{\Pr}_\theta(y|x) \tag{19}$$

where $\bar{\Pr}_\theta(y|x)$ represents the average prediction across augmented copies.

Laplacian Adjusted Maximum-likelihood Estimation (LAME) [71] takes a fundamentally different approach. Unlike TENT and MEMO, which update internal parameters, LAME adapts the model's output probabilities. It finds the optimal latent assignments $\tilde{z}_i$ for each test sample through minimizing a regularized likelihood objective given by the Kullback-Leibler divergences between latent assignments and source model predictions, subject to probability simplex constraints:

$$\mathcal{L}_{\text{LAME}}(\tilde{Z}) = \sum_{i} \text{KL}(\tilde{z}_i || q_i) - \sum_{i,j} w_{\text{ij}} \tilde{z}_i^T \tilde{z}_j \tag{20}$$

q_i represents the source model's predictions and w_{ij} measures feature similarity between samples i and j. The Laplacian regularization term encourages neighbouring points in feature space to have consistent class assignments, leveraging the manifold smoothness assumption [72] widely used in semi-supervised learning. To speed up the optimization of 20, LAME utilises a concave-convex procedure (CCCP) [73], which guarantees convergence by iteratively replacing the concave Laplacian term with its linear approximation.

Forward-Optimization Adaptation (FOA) [74] introduces a derivative-free adaptation method specifically designed for resource-constrained deployment environments that cannot support backpropagation operations. It addresses scenarios where gradient computation is infeasible, such as quantized models or systems with hardcoded parameters on specialized hardware. The method operates through two different mechanisms: prompt-based input adaptation and activation-level feature alignment.

FOA uses Covariance Matrix Adaptation Evolution Strategy (CMA-ES) to optimize prompts without gradients. At each iteration, CMA-ES samples a population of candidate prompts from a multivariate normal distribution, evaluates their fitness, and updates the distribution parameters based on the ranking of solutions. Additionally, FOA implements a ''back-to-source'' activation shifting mechanism that directly adjusts final-layer features by moving them toward the source domain center: $e'_N \leftarrow e_N + \gamma d_t$. Where $d_t = \mu_N^S - \mu_N(t)$ represents the dynamic shifting direction updated via exponential moving averages, and e_i are classification tokens from the ith layer of the ViT.

Traditional CMA approaches are infeasible in a constrained TTA setting as they typically rely on ground truth, or target a too high-dimensional parameter space [74]. To account for this, FOA learns a set of input prompt embeddings $\boldsymbol{p} \in \mathbb{R}^{d \times N_p}$ that are inserted before the model input rather than updating model parameters directly. The optimization objective aims to find optimal prompts by minimizing a fitness function that combines prediction entropy with feature distribution alignment as:

$$\mathcal{L}(f_\Theta(\boldsymbol{p}; \mathcal{X}_t)) = -\sum_{x \in \mathcal{X}_t} \sum_{c \in \mathcal{C}} \hat{y}_c log(\hat{y}_c) + \lambda \sum_{i=1}^{N} ||\boldsymbol{\mu}_t(\mathcal{X}_t) - \boldsymbol{\mu}_i^S||_2 + ||\boldsymbol{\sigma}_i(\mathcal{X}_t) - \boldsymbol{\sigma}_i^S||_2 \tag{21}$$

where *boldsymbolp* is prompts, λ is a hyperparameter representing the trade-off between prediction entropy and feature distribution.

In an alternative approach, CoTTA [75] employs a teacher-student framework with weighted average pseudo-labels to improve adaptation stability. Initially, the teacher network is set to be the same as the source pre-trained network. The teacher network produces the pseudo-label and the student network is then updated by the cross entropy loss between the student and teacher predictions. After updating the student model $\theta_t \rightarrow \theta_{t-1}$, the weights of the teacher model (θ') are updated using the exponential moving average of the student's weights:

$$\theta'_{t+1} = \alpha \theta'_t + (1 - \alpha)\theta_{t+1} \tag{22}$$

EcoTTA [76] improves memory consumption, and long-term adaptation stability compared to CoTTA by combining architectural modifications with regularization. The method involves partitioning a pre-trained model encoder into K parts and attaches lightweight ''meta networks'' to

each partition. Each meta network consists of only one batch normalization layer and one convolution block. During adaptation, the pre-trained network remains frozen and only the meta networks are updated, eliminating the need to store intermediate activations for the original parameters.

The partitioning strategy densely divides shallow layers compared to deeper layers, allowing meta networks to modify earlier feature representations while preserving semantic knowledge in the frozen deeper layers. For the main adaptation task, EcoTTA uses entropy minimization with sample filtering, where only samples with entropy below a predefined threshold (H_0) are adapted.

$$\mathcal{L}^{\text{ent}} = \mathbb{I}_{\{H(\hat{y})<H_0\}} \cdot H(\hat{y}) \tag{23}$$

To prevent catastrophic forgetting, EcoTTA proposes self-distilled regularization which constrains the meta network outputs $\tilde{x}_k$ to remain close to the frozen network outputs x_k using L1 distance:

$$R^k_{\theta_k} = \|\tilde{x}_k - x_k\|_1 \tag{24}$$

$$\mathcal{L}^{\text{total}}_{\theta} = \mathcal{L}^{\text{ent}}_{\theta} + \lambda \sum_{k}^{K} R^k_{\theta_k} \tag{25}$$

The combination of both the entropy and regularization loss balances adaptation through entropy minimization with source knowledge preservation through regularization.

FoCTTA [77] further addresses the memory efficiency challenges in continual test time adaptation by focusing on adaptation-critical representation layers, rather than updating all batch normalization layers. This method shifts from updating batch norm layers to selectively updating representation layers, which are more resilient to small batch sizes.

This approach uses an offline warm-up training phase after pre-training where augmentations are applied to the source data to simulate distributional shifts. The average gradient norm of each layer is used to quantify the importance of that layer in adaptation, where the layers with the largest average gradient norms are considered the most important. The selection of the layers is conducted by $\alpha\|s\|$, where α is a tuneable hyperparameter to balance memory cost and accuracy and s is a vector of the importance of all layers in the pre-trained model. At test time, only the selected adaptation-critical layers are optimized.

D. TEST-TIME TRAINING

Test-Time Training (TTT) [78] introduces an approach to test-time adaptation that modifies model parameters at inference time using a self-supervised auxiliary task. The model architecture follows a y-structure with a shared feature extractor θ_e and two task specific branches: θ_m for the main task branch and θ_s for the self-supervised task. In [78], the authors utilize rotation prediction as the auxiliary task however it is extended upon in [79] where image reconstruction is selected as the auxiliary task.

Initially the model is trained on both tasks, where the joint training objective is a combination of the loss for the self-supervised and main task:

$$\min_{\theta_e,\theta_m,\theta_s} \frac{1}{n}\sum_{i=1}^{n} l_m(x_,, y_i; \theta_m, \theta_e) + l_s(x_i; \theta_s, \theta_e) \tag{26}$$

At test time, TTT performs adaptation by fine tuning the shared feature extractor θ_e by minimizing the auxiliary task loss:

$$\min_{\theta_e} l_s(x; \theta_s, \theta_e) \tag{27}$$

The model makes a prediction using the updated parameters $\theta(x) = (\theta^*_e, \theta_m)$. The online variant of test-time training extends on this by instead, at a test sample x_t, initializing θ with $\theta(x_{t-1})$ which was updated on the sample x_{t-1}.

ReC-TTT [80] employs contrastive feature reconstruction for adaptation. Unlike the previously mentioned TTT methods, ReC-TTT operates through a multi-encoder architecture consisting of two trainable encoders $f_{\theta 1}$ and $f_{\theta 2}$, a pre-trained frozen decoder $f_{\theta F}$, and a shared decoder g_θ. The method implements cross-reconstruction learning, where the decoder learns to reconstruct features from the frozen encoder using features from the trainable encoders and vice versa. The auxiliary loss uses global cosine similarity between features at different layers:

$$L_{\text{aux}} = \sum_{l=1}^{L} 1 - \frac{\langle \text{sg}(f^l_E), f^l_D \rangle}{\text{sg}(\|f^l_E\|_2)\|f^l_D\|_2} \tag{28}$$

where sg represents the stop gradient operation to prevent direct gradient propagation into the encoder, and f^l_E and f^l_D represent the flattened features at the lth layer of the encoder and decoder respectively.

During the training stage, the network is jointly trained on the supervised main task and the contrastive reconstruction minimizing a combination of the loss for the auxiliary task and main task similarly to [78]. In addition to this, the second trainable encoder receives an augmented image, in the case of this implementation the augmentation applied is a horizontal flip. To align the distributions predicted by the two encoders, a consistency loss is applied by measuring Kullback-Leibler divergence between predictor outputs. At test time, the decoder remains frozen while the trainable encoders are updated using the auxiliary loss. Encoder weights are reinitialized for each test batch, and final predictions combine outputs from both encoders.

E. COMPARISON

In order to evaluate the suitability of different approaches for resource-constrained environments, we present a comparison across key dimensions that directly impact IoT feasibility. Tables 2 and 3 provide detailed analysis of the surveyed adaptation methods. The comparison framework evaluates methods across several dimensions relevant to resource-constrained environments and aids the overall discussion on suitability for IoT deployment in section V-F. Table 5 indicating general use cases and examples can be found in appendix A.

TABLE 2. Summary of concept drift adaptation methods surveyed in this paper. Computational requirements are classified as: Very Low - Microcontroller-suitable (minimal overhead); Low - Edge device-suitable (light requirements); Moderate - Dedicated hardware needed; High - GPU required; Very High - Multi-GPU setup needed. *Can be extended to unsupervised.

Method name	Category	Computational requirements	Implementation Hardware	Network Type
Prediction-Time Batch Normalization [68]	Unsupervised	Low Training Low at inference time	-	NN with BN layers
Covariate Shift Adaptation by modifying BN statistics [67]	Unsupervised	Low Training Low at Inference time	Nvidia RTX 2080 GPUs (1-8 units) or Nvidia V100 GPUs (1-4 units)	NN with BN layers
Test-Time Training [78]	Self-Supervised	Low training Moderate at inference time	-	CNN
FOA [74]	Unsupervised	High training High at inference time	-	ViT
MEMO [70]	Self-Supervised	Low training High at inference time	4 Intel Xeon Skylake 6130 CPUs 1 NVIDIA TITAN RTX GPU	Architecture agnostic
LAME [71]	Unsupervised	Very Low training Low at inference time	-	Architecure agnostic
TENT [69]	Unsupervised	Low training Moderate at test time	-	NNs with BN layers
TWIN-ADAPT [51]	Supervised*	Low training Low at test time	-	Gradient Boosting Decision Tree
TML-CD [3]	Supervised	Very Low training Very Low at test time	STM32H743 (1024KB RAM) STM32F767 (512KB RAM) STM32F401 (96KB RAM)	CNN feature extractor + k-NN
TinyOL [50]	Supervised	Very Low training Very Low at test time	Arduino Nano 33 BLE Sense (Arm Cortex-M4 @ 64MHz, 256KB SRAM)	Any NN + additional layer
CoTTA [75]	Unsupervised	Low training High at test time	-	Architecture agnostic
FoCTTA [77]	Unsupervised	Low training Moderate at test time	1 Nvidia 3090	CNN

The computational requirements are categorized from ''Very Low'' to ''Very High'' based on the training complexity and inference-time overhead. Training requirements consider whether methods need additional pre-training phases, auxiliary task learning, or complex search procedures. Inference requirements focus on the computational burden during actual deployment, including gradient computations, multiple forward passes, or parameter updates.

F. DISCUSSION

Given the aforementioned methods, we now discuss the feasibility of various strategies on IoT platforms. Importantly, networks are assumed to initially be trained externally prior to deployment on resource-constrained devices. As such, only activation resource consumption as well as resource requirements during updates are considered.

Batch normalization approaches are generally of low complexity and do not require labels, which makes them ideal candidates for unsupervised drift adaptation. Moreover, batch normalization techniques, such as AdaBN [81], have been successful when applied to UDA problems. However, batching inherently breaks the assumption that the input is a data stream, necessitating additional care when constructing the computational pipeline to ensure the adaptation strategy is compatible with the detection. Furthermore, batch normalization assumes $p(x|y) = q(x|y)$ and, thus, it does not account for changes in the learned conditional, only in the prior and marginal distributions [68]. Lastly, batch normalization only works to remove changes in the first- and second-order moments [67], i.e., changes in skew and kurtosis are not taken into account. Batching also introduces additional memory and latency overheads as several data points are stored in memory.

TTA frameworks based on student-teacher optimisation, such as CoTTA, are bound by the need to run a larger network during adaptation, thereby drastically increasing the computational requirements. Moreover, without re-training the teacher model, there is no guarantee that the drift will not similarly degrade its accuracy and, consequently, it is not ideal in a labelless context. Other TTA approaches, like FOA, appear promising, but without a fog-node it is still infeasible to deploy a ViT-based model for IoT devices as it still necessitates memory far outside the bounds of a GPU-less IoT device, reporting an inference time peak memory of 208MB with 8-bit quantization [74].

Although NAS provides good accuracy, the high computational complexity associated with training numerous networks during search makes it unsuitable for adaptation on constrained devices. In fact, designing a new NAS for each problem or hardware platform is extremely

TABLE 3. Continuation: summary of concept drift adaptation methods surveyed in this paper. Computational requirements are classified as: Very Low - Microcontroller-suitable (minimal overhead); Low - Edge device-suitable (light requirements); Moderate - Dedicated hardware needed; High - GPU required; Very High - Multi-GPU setup needed. *Can be extended to unsupervised.

Method name	Category	Computational requirements	Implementation Hardware	Network Type
ReC-TTT [80]	Self-Supervised	Moderate training High at test time	NVIDIA A100	CNN
EcoTTA [76]	Unsupervised	Low training Low at test time	Energy-constrained evaluation platforms	CNN
HardCoRe-NAS [54]	N/A	High training Low at inference time	Training: 8 NVIDIA V100 408 +15 N GPU hours Evaluation: Intel Xeon CPU, NVIDIA P100 GPU	CNN
MSNAS [61]	N/A	Moderate training Very Low at inference time	Training: 1 NVIDIA GTX 1080 8 GPU hours Evaluation: 100-320kb SRAM 256KB-1MB Flash storage	CNN
Co-Design NAS [62]	N/A	High training Low at inference time	Target Platform: Altera Cyclone IV FPGA, 100 MHz clock	CNN
E-DNAS [56]	N/A	Very High Training Low at inference time	Training: 8 × NVIDIA Tesla V100 GPUs Target Platform: TI TDA2x SoC (C66x DSP + ARM Cortex-A15 + EVE)	CNN
µNAS [59]	N/A	Very High Training Very Low at inference time	Target: 64KB Flash, 64KB SRAM, Low-frequency, single-core	CNN
HIO-NAS [63]	N/A	Very high training Moderate at test time	Training: 2 NVIDIA GeForce RTX 3090 GPUs (24GB each), Intel Core i9-10940X CPU, 128GB RAM Evaluation: Intel Core i7-6700 CPU (3.40 GHz), 32GB DDR4 RAM	CNN

TABLE 4. Available benchmarks on selected data sets. Acc - accuracy (prequential), DA - detection accuracy, DD - detection delay, FPR - false positive rate, MDR - missed detection ratio, MTD - mean time to detection.

	electricity	forest cover		Insects		AbruptInsects	adult			bank				Custom dataset		
	Acc	Acc	DA	Acc	DA	DA	DD	FPR	DA	Acc	DD	FPR	DA	MDR	FPR	MTD
ADWIN	81.33 [38]	80.48 [38]												0.227 [85]	0.117 [85]	675.03 [85]
PH														0.087 [85]	0.277 [85]	624.07 [85]
IKS			0.85 [43]	79.203 [28]	0.78 [43]	0.85 [43]	148.40 ±123.00 [86]	0.15 [86]	0.80±0.42 [86]		228.80±158.23 [86]	0.03 [86]	0.80±0.42 [86]			
Driftlens																
D3	86.69 [38], 85.3 [42]	87.17 [38]		0.6786 [42]												
HDDDM							5.00 [86]	0.09 [86]	1.00 [86]	89.6 [31]	10.00±1.41 [86]	0.05 [86]	1.00 [86]			
NN-DVI	84.78 [37]															
STUDD			0.68 [43]		0.84 [43]	0.83 [43]										
ODIN																
MD3										87.35 [31]						

resource-inefficient. In response, some work has been done to alleviate this problem, for example, the once-for-all (OFA) network [64] avoids retraining by training specialized sub-networks once, which can be directly deployed. However, this assumes a static distribution and, consequently, does not ward against concept drift. Moreover, weight sharing strategies can also lead to inaccurate performance estimation when selecting candidate sub-networks [63]. Another commonality amongst NAS-strategies is their dependence on labels, which further disincentivizes their use in unsupervised concept drift strategies, but they could be a powerful alternative for off-device few-shot strategies. Alternatively, NAS-strategies

TABLE 5. Overview of adaptation approaches across hardware and use cases.

	Adaptation Type	Target Hardware Profile	Generalised Use Case	Example IoT Use Case
TinyOL	Online Learning	MCU	Simple, Monomodal Data with External Detector	Real-time drift handling on streaming single-sensor data
TML-CD	Online Learning	MCU	Simple Classification	Concept drift in lightweight classification tasks
VAE4AS	Online Learning	Edge CPU	Anomaly Detection	Unsupervised anomaly detection on continuous industrial sensor feeds
TWIN-ADAPT	Online Learning	Edge CPU	Highly Controlled Environments	Digital-twin based feedback in smart labs
MSNAS	NAS	MCU	Model Optimization for IoT	Generating ultra-compact TinyML models for IoT endpoints
HardCoRe-NAS	NAS	Edge CPU / Edge GPU	Model Optimization for Latency Constrained Systems	Offline architecture search for models on edge inference devices
HIO-NAS	NAS	Edge SoC / Edge GPU	Model Optimization for FPGAs	Hardware-aware architectures for FPGA/SoC accelerators in industrial AI
CoDesign-NAS	NAS	Edge SoC	Model & Hardware Co-Optimization	Co-optimized architectures for AI accelerators in embedded robotics
Test-Time Training	Test-Time Adaptation	Edge GPU / Cloud	Self-supervised feature correction when computational cost is less of a concern	Self-supervised feature correction in mobile robots or autonomous vehicles facing unseen environments
Prediction-Time BN	Test-Time Adaptation	Edge GPU	Batch-level robustness adaptation in high-throughput vision pipelines	Batch-level robustness adaptation in vision pipelines
FoCTTA	Test-Time Adaptation	Edge SoC / Edge GPU	Embedded Vision for Mobile Platforms	Memory-efficient drift response for embedded vision in drones of mobile platforms
MEMO	Test-Time Adaptation	Edge SoC	One-shot Adaptation Against Sudden Drift	Single-sample correction for sudden visual corruption in smart surveillance
FOA	Test-Time Adaptation	Edge CPU / Edge SoC	Inexpensive ViT Adaptation	Gradient-free adaptation for quantized ViT models on smartphones, drones, or AR devices

could be used in conjunction with other methods to provide a more optimal starting architecture and baseline accuracy.

In general, deep learning networks often suffer from high computational complexity. Ignoring bias, convolutions, for example, have a memory footprint of:

$$c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o \cdot c_o \tag{29}$$

that is, it scales linearly as a combination of input/output channels, output width and height, and kernel size. Moreover, convolutional networks often feature several activation functions, which are known to be computationally expensive [82]. That being said, some architectures are inherently simpler to compress than others, for example, Lin, et al. [83] introduce the concept of patch-based inference, which significantly reduces peak memory of CNNs by operating on smaller spatial regions of the feature map at a time. In [84], the authors use patch-based inference in combination with other memory-saving concepts, such as sparse updates, to successfully demonstrate CNN training on an IoT device. Autoencoders have also been shown to be strong candidate networks when considering un- and self-supervised concept drift adaptation due to their flexibility.

Other networks, such as temporal ones, are inherently more difficult to compress to the same extent as their primary footprint comes from their internal memory. Similarly, while efficient transformers is the current focus of a large portion of the embedded machine learning research community, the inherent cost of multi-headed attention makes it unlikely that

IoT devices will be able to adopt them under their current constraints.

Another notion to keep in mind when designing a concept drift adaptation strategy is the interplay between detection and adaptation strategies. For example, while PH can theoretically use any metric, it is typical to focus on mean and variance. Consequently, it will be a poor choice when working with a batch-normalising strategy. Moreover, if your adaptation method utilizes online learning, it is beneficial to make use of stream-based detection methods. Lastly, as SRAM is a heavily constrained resource, utilizing memory-based detection methods inherently necessitates a less memory-intensive adaptation strategy. Memory-based methods have the additional downside of needing to pre-determine the right memory size. One can also make it adaptive, but that further increases complexity leading to strain on the already limited SRAM in IoT devices.

VI. FUTURE WORK

With increasingly fast advances in the field of machine learning and IoT technology, this topic offers many opportunities for further investigation.

Absent from this paper is a discussion on relative performance, especially for detection methods. While a comparative benchmarking table was prepared, a snippet of which is found in appendix A, there is almost no overlap between data sets and metrics used by various authors. Consequently, a direct comparison of strategies across different data sets could be more misleading than informative. While out of scope for this work, there is clear room for such a comparative work.

VII. CONCLUSION

The investigation of autonomous concept drift detection and adaptation for IoT and resource-constrained platforms reveals a persistent trade-off between detection accuracy, adaptability, and computational feasibility. While lightweight statistical detectors, such as ADWIN and PH, remain attractive for their minimal resource demands, their inability to capture complex, multivariate drifts limits practical utility in dynamic environments. Conversely, model-assisted and discriminative approaches, such as ODIN, STUDD, and D3, deliver state-of-the-art performance but incur notable memory overheads that may exceed the capacity of severely constrained devices, particularly when auxiliary models or dimensionality reduction mechanisms are required. Similarly, specialized algorithms like NN-DVI and MD3 demonstrate theoretical promise but often fail to achieve practical efficiency or broad applicability under embedded constraints.

On the adaptation front, batch normalization techniques, exemplified by AdaBN, offer an elegant and relatively low-cost solution for covariate shift; however, their reliance on batching, limited ability to address higher-order distributional changes, and assumptions regarding conditional stability reduce their generality. Test-time adaptation strategies, such as CoTTA and FOA, present compelling accuracy gains but remain resource-prohibitive for on-device execution, especially in the absence of hierarchical or fog-assisted infrastructure. NAS approaches and efficient architectural designs, including Once-for-All networks and patch-based inference (e.g., MCUNetV2), signal progress toward on-device adaptability, yet their current limitations - chiefly the dependence on labels, computationally expensive search processes, and static-distribution assumptions - underscore the necessity of further innovation. Emerging paradigms in efficient model compression, online learning, and sparse update strategies illustrate the feasibility of certain deep learning architectures in IoT contexts, albeit with significant engineering challenges.

Collectively, these findings emphasize that the suitability of a given detection or adaptation strategy is deeply context-dependent. Factors such as the nature of the data stream, expected drift patterns, available memory, energy budgets, and latency requirements fundamentally shape the design space. As such, the findings of this paper cannot conclude on a universal best solution, but instead aim to guide future research and implementation by highlighting the strengths, weaknesses and use-cases for a diverse set of models.

APPENDIX

A. BENCHMARKS ON DETECTION ALGORITHMS

See Table 4.

B. ALGORITHMIC COMPLEXITIES FOR DETECTION METHODS

In the following we present the time and space complexities for some of the detection methods presented in earlier sections. These complexities are either taken from the original papers or derived from known complexities of underlying concepts. Hard proofs are up to the reader. The complexities are formulated in terms of the number of attributes m, the number of samples in a window or batch n, and, in some cases, the number of data points in the entire previous history N.

For ADWIN, we consider the optimised version ADWIN2, which has a time complexity of $O(m \log^2 n)$ and space complexity of $O(m \log n)$ [1].

PH and CUSUM technically operate in constant time $O(1)$ for both space and time, as they are performing only a constant set of simple arithmetic operations in each step. However, if we consider monitoring multiple incoming attributes, these computations will have to be performed for each of them individually, leaving us with a complexity that is linear in the number of attributes, $O(m)$.

IKS mentions the time complexity of a tree update as $O(\log n)$ [28], again for multivariate data streams each attribute would be monitored individually, causing a time complexity of $O(m \log n)$. The auxiliary space occupied by a Cartesian tree (treap) is $O(n)$ [87], leading to a final space complexity of $O(nm)$.

Driftlens employs a PCA reduction in each window [7], neglecting the mapping from the raw input into the

embedding space we can assume the same time complexity as a PCA reduction, $O(nm^2 + n^3)$ [88].

D3 retrains and prompts an additional classifier in each window. Since the complexity of this method depends strongly on the type of classifier chosen, we take inspiration from the original paper [38] and consider the case of a simple logistic regression. This classifier has a training time and space complexity of $O(nm)$.

In every batch, HDDDM computes the Hellinger distance between two histograms with $\sqrt{n}$ bins for each attribute [36]. The time and space complexities are $O(m\sqrt{n})$.

For NN-DVI, the authors provide a model construction complexity when using kd-trees as $O(kn \log n)$ [37], since we are not interested in hyperparameters, we consider the complexity to be $O(n \log n)$.

STUDD performs the detection using PH in constant time, $O(1)$, and only constant $O(1)$ spatial overhead is caused by the detection process alone [35]. In this case, we have to remark on the additional model stored on device (the student), but that is outside of algorithmic complexity considerations.

PCA-DD relies similarly to Driftlens on PCA performed every iteration, leading to a complexity of $O(nm^2 + n^3)$ [41], [88].

For ODIN, the authors don't provide specific instructions on the implementation of the distribution approximations, overall we can assume that there is a set of bins for each cluster. Any incoming point causes the centroid to be recomputed as well as the distance distribution and the density bands. The latter naively requires a sorting of the existing bins in $O(c \log c)$ assuming there are c points in a cluster. However, the ODIN framework in its clusters and density bands retains the information about the embeddings of all previously seen data points. We therefore assume a worst-case scenario time complexity of $O(N \log N)$ and space complexity of $O(N)$, subject to potential optimisations not included in the original paper [33].

Lastly, MD3 is a fairly simple method that requires only constant space and time $O(1)$ to compute margins or blind spot density.

C. DEPLOYMENT SCENARIOS FOR ADAPTATION METHODS

See Table 5.

ACKNOWLEDGMENT

(Ask Espensønn Øren, Nina Kuna Peuker, and Akira-Miranda Adeyomi Adeniran-Lowe contributed equally to this work.) ChatGPT has been used for grammatical assistance and enhancement, in particular for the abstract, introduction, and conclusion.

REFERENCES

[1] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining*, Apr. 2007, pp. 443–448.

[2] I. Frias-Blanco, J. D. Campo-Avila, G. Ramos-Jimenez, R. Morales-Bueno, A. Ortiz-Diaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 3, pp. 810–823, Mar. 2015.

[3] S. Disabato and M. Roveri, "Tiny machine learning for concept drift," 2021, *arXiv:2107.14759*.

[4] R. N. Gemaque, A. F. J. Costa, R. Giusti, and E. M. Dos Santos, "An overview of unsupervised drift detection methods," *WIREs Data Mining Knowl. Discovery*, vol. 10, no. 6, p. 1381, Nov. 2020.

[5] L. Yuan, F. Ye, W. Zhou, W. Yuan, and X. You, "Concept drift adaptation with scarce labels: A novel approach based on diffusion and adversarial learning," *Eng. Appl. Artif. Intell.*, vol. 137, Nov. 2024, Art. no. 109105.

[6] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surveys*, vol. 46, no. 4, pp. 1–37, Apr. 2014.

[7] S. Greco, B. Vacchetti, D. Apiletti, and T. Cerquitelli, "Unsupervised concept drift detection from deep learning representations in real-time," 2024, *arXiv:2406.17813*.

[8] H. Yu, T. Liu, J. Lu, and G. Zhang, "Automatic learning to detect concept drift," 2021, *arXiv:2105.01419*.

[9] G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter, "Continual lifelong learning with neural networks: A review," 2018, *arXiv:1802.07569*.

[10] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," 2016, *arXiv:1612.00796*.

[11] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell, "Progressive neural networks," 2016, *arXiv:1606.04671*.

[12] A. Soltoggio, "Short-term plasticity as cause–effect hypothesis testing in distal reward learning," *Biol. Cybern.*, vol. 109, no. 1, pp. 75–94, Feb. 2015.

[13] S. M. Ahmed, D. S. Raychaudhuri, S. Oymak, and A. K. Roy-Chowdhury, "Source distribution weighted multisource domain adaptation without access to source data," in *Handbook of Statistics*, vol. 48. Amsterdam, The Netherlands: Elsevier, 2023, pp. 81–105.

[14] J. Li, K. Malialis, C. G. Panayiotou, and M. M. Polycarpou, "Unsupervised incremental learning with dual concept drift detection for identifying anomalous sequences," 2024, *arXiv:2403.03576*.

[15] R. Chalapathy and S. Chawla, "Deep learning for anomaly detection: A survey," 2019, *arXiv:1901.03407*.

[16] Z. Niu, K. Yu, and X. Wu, "LSTM-based VAE-GAN for time-series anomaly detection," *Sensors*, vol. 20, no. 13, p. 3738, Jul. 2020.

[17] D. Fährmann, N. Damer, F. Kirchbuchner, and A. Kuijper, "Lightweight long short-term memory variational auto-encoder for multivariate time series anomaly detection in industrial control systems," *Sensors*, vol. 22, no. 8, p. 2886, Apr. 2022.

[18] M. Jaworski, L. Rutkowski, and P. Angelov, "Concept drift detection using autoencoders in data streams processing," in *Proc. Int. Conf. Artif. Intell. Soft Comput.*, 2020, pp. 124–133.

[19] R. S. M. Barros and S. G. T. C. Santos, "A large-scale comparison of concept drift detectors," *Inf. Sci.*, vols. 451–452, pp. 348–370, Jul. 2018.

[20] S. Agrahari and A. K. Singh, "Concept drift detection in data stream mining: A literature review," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 34, no. 10, pp. 9523–9540, Nov. 2022.

[21] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," 2020, *arXiv:2004.05785*.

[22] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Proc. Brazilian Symp. Artif. Intell.*, 2004, pp. 286–295.

[23] M. Baena-García, J. D. Campo-Ávila, R. Fidalgo-Merino, A. Bifet, R. Gavaldà, and R. Morales-Bueno, "Early drift detection method," in *Proc. 4th Int. Workshop Knowl. Discovery Data Streams*, 2006, pp. 77–86.

[24] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, nos. 1–2, p. 100, Jun. 1954.

[25] D. V. Hinkley, "Inference about the change-point from cumulative sum tests," *Biometrika*, vol. 58, no. 3, pp. 509–523, Dec. 1971.

[26] S. Wares, J. Isaacs, and E. Elyan, "Data stream mining: Methods and challenges for handling concept drift," *Social Netw. Appl. Sci.*, vol. 1, no. 11, p. 1412, Nov. 2019.

[27] G. Ditzler, M. Roveri, C. Alippi, and R. Polikar, "Learning in nonstationary environments: A survey," *IEEE Comput. Intell. Mag.*, vol. 10, no. 4, pp. 12–25, Nov. 2015.
[28] D. M. Dos Reis, P. Flach, S. Matwin, and G. Batista, "Fast unsupervised online drift detection using incremental Kolmogorov–Smirnov test," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 1545–1554.
[29] Y. Dodge, "Kolmogorov–Smirnov Test," in *The Concise Encyclopedia of Statistics*. New York, NY, USA: Springer, 2008, pp. 283–287.
[30] P. Lindstrom, B. Mac Namee, and S. J. Delany, "Drift detection using uncertainty distribution divergence," *Evolving Syst.*, vol. 4, no. 1, pp. 13–25, Mar. 2013.
[31] T. S. Sethi and M. Kantardzic, "On the reliable detection of concept drift from streaming unlabeled data," *Expert Syst. Appl.*, vol. 82, pp. 77–99, Oct. 2017.
[32] M. Hearst, S. Dumais, E. Osuna, J. Platt, and B. Scholkopf, "Support vector machines," *IEEE Intell. Syst. Appl.*, vol. 13, no. 7, pp. 18–28, Jul. 1998.
[33] A. Suprem, J. Arulraj, C. Pu, and J. Ferreira, "ODIN: Automated drift detection and recovery in video analytics," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 2453–2465, Aug. 2020.
[34] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, Mar. 1951.
[35] V. Cerqueira, H. M. Gomes, A. Bifet, and L. Torgo, "STUDD: A student–teacher method for unsupervised concept drift detection," *Mach. Learn.*, vol. 112, no. 11, pp. 4351–4378, Nov. 2023.
[36] G. Ditzler and R. Polikar, "Hellinger distance based drift detection for nonstationary environments," in *Proc. IEEE Symp. Comput. Intell. Dyn. Uncertain Environments (CIDUE)*, Apr. 2011, pp. 41–48.
[37] D. Shang, G. Zhang, and J. Lu, "Novelty-aware concept drift detection for neural networks," *Neurocomputing*, vol. 617, Feb. 2025, Art. no. 128933.
[38] Ö. Gözüaçık, A. Büyükçakır, H. Bonab, and F. Can, "Unsupervised concept drift detection with a discriminative classifier," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, Nov. 2019, pp. 2365–2368.
[39] L. I. Kuncheva and W. J. Faithfull, "PCA feature extraction for change detection in multidimensional unlabeled data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 25, no. 1, pp. 69–80, Jan. 2014.
[40] A. A. Qahtan, B. Alharbi, S. Wang, and X. Zhang, "A PCA-based change detection framework for multidimensional data streams: Change detection in multidimensional data streams," in *Proc. 21st ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2015, pp. 935–944.
[41] O. Abdel Wahab, "Intrusion detection in the IoT under data and concept drifts: Online deep learning approach," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19706–19716, Oct. 2022.
[42] D. Lukats, O. Zielinski, A. Hahn, and F. Stahl, "A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams," *Int. J. Data Sci. Analytics*, vol. 19, no. 1, pp. 1–31, Jan. 2025.
[43] E. Werner, N. Kumar, M. Lieber, S. Torge, S. Gumhold, and W. E. Nagel, "Towards computational performance engineering for unsupervised concept drift detection—Complexities, benchmarking, performance analysis," 2023, *arXiv:2304.08319*.
[44] G. Widmer and M. Kubat, "Learning in the presence of concept drift and hidden contexts," *Mach. Learn.*, vol. 23, no. 1, pp. 69–101, Apr. 1996.
[45] J. Liu, X. Li, and W. Zhong, "Ambiguous decision trees for mining concept-drifting data streams," *Pattern Recognit. Lett.*, vol. 30, no. 15, pp. 1347–1355, Nov. 2009.
[46] J. Wilson, S. Chaudhury, and B. Lall, "Homogeneous–heterogeneous hybrid ensemble for concept-drift adaptation," *Neurocomputing*, vol. 557, Nov. 2023, Art. no. 126741.
[47] K. Malialis, M. Roveri, C. Alippi, C. G. Panayiotou, and M. M. Polycarpou, "A hybrid active-passive approach to imbalanced nonstationary data stream classification," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Singapore, Dec. 2022, pp. 1021–1027.
[48] S.-C. You and H.-T. Lin, "A simple unlearning framework for online learning under concept drifts," in *Proc. Adv. Knowl. Discovery Data Mining*, 2016, pp. 115–126.
[49] A. Artelt, K. Malialis, C. G. Panayiotou, M. M. Polycarpou, and B. Hammer, "Unsupervised unlearning of concept drift with autoencoders," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2023, pp. 703–710.
[50] H. Ren, D. Anicic, and T. A. Runkler, "TinyOL: TinyML with online-learning on microcontrollers," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 1–8.
[51] R. Gupta, B. Tian, Y. Wang, and K. Nahrstedt, "TWIN-ADAPT: Continuous learning for digital twin-enabled online anomaly classification in IoT-driven smart labs," *Future Internet*, vol. 16, no. 7, p. 239, Jul. 2024.
[52] U. V. Luxburg, I. Guyon, S. Bengio, H. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, "Neural information processing systems foundation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4–9.
[53] I. Salehin, M. S. Islam, P. Saha, S. M. Noman, A. Tuni, M. M. Hasan, and M. A. Baten, "AutoML: A systematic review on automated machine learning with neural architecture search," *J. Inf. Intell.*, vol. 2, no. 1, pp. 52–81, Jan. 2024.
[54] N. Nayman, Y. Aflalo, A. Noy, and L. Zelnik-Manor, "HardCoRe-NAS: Hard constrained diffeRentiable neural architecture search," 2021, *arXiv:2102.11646*.
[55] S. Lacoste-Julien, M. Jaggi, M. Schmidt, and P. Pletscher, "Block-coordinate frank-wolfe optimization for structural SVMs," 2012, *arXiv:1207.4747*.
[56] J. G. López, A. Agudo, and F. Moreno-Noguer, "E-DNAS: Differentiable neural architecture search for embedded systems," in *Proc. 25th Int. Conf. Pattern Recognit. (ICPR)*, Jan. 2021, pp. 4704–4711.
[57] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, and K. Keutzer, "FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search," 2018, *arXiv:1812.03443*.
[58] T.-J. Yang, A. Howard, B. Chen, X. Zhang, A. Go, M. Sandler, V. Sze, and H. Adam, "NetAdapt: Platform-aware neural network adaptation for mobile applications," 2018, *arXiv:1804.03230*.
[59] E. Liberis, Ł. Dudziak, and N. D. Lane, "μNAS: Constrained neural architecture search for microcontrollers," 2020, *arXiv:2010.14246*.
[60] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," 2018, *arXiv:1802.01548*.
[61] H.-P. Cheng, T. Zhang, Y. Yang, F. Yan, H. Teague, Y. Chen, and H. Li, "MSNet: Structural wired neural architecture search for Internet of Things," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop (ICCVW)*, Oct. 2019, pp. 2033–2036.
[62] Q. Lu, W. Jiang, X. Xu, Y. Shi, and J. Hu, "On neural architecture search for resource-constrained hardware platforms," 2019, *arXiv:1911.00105*.
[63] O. T.-C. Chen, Y.-X. Chang, C.-Y. Chung, Y.-Y. Cheng, and M.-H. Ha, "Hardware-aware iterative one-shot neural architecture search with adaptable knowledge distillation for efficient edge computing," *IEEE Access*, vol. 13, pp. 54204–54222, 2025.
[64] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," 2019, *arXiv:1908.09791*.
[65] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
[66] J. Liang, R. He, and T. Tan, "A comprehensive survey on test-time adaptation under distribution shifts," *Int. J. Comput. Vis.*, vol. 133, no. 1, pp. 31–64, Jan. 2025.
[67] S. Schneider, E. Rusak, L. Eck, O. Bringmann, W. Brendel, and M. Bethge, "Improving robustness against common corruptions by covariate shift adaptation," 2020, *arXiv:2006.16971*.
[68] Z. Nado, S. Padhy, D. Sculley, A. D'Amour, B. Lakshminarayanan, and J. Snoek, "Evaluating prediction-time batch normalization for robustness under covariate shift," 2020, *arXiv:2006.10963*.
[69] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," 2020, *arXiv:2006.10726*.
[70] M. Zhang, S. Levine, and C. Finn, "MEMO: Test time robustness via adaptation and augmentation," 2021, *arXiv:2110.09506*.
[71] M. Boudiaf, R. Mueller, I. B. Ayed, and L. Bertinetto, "Parameter-free online test-time adaptation," 2022, *arXiv:2201.05718*.
[72] S. Theodoridis and K. Koutroumbas, "Feature generation I: Data transformation and dimensionality reduction," in *Pattern Recognition*, 4th ed., Boston, MA, USA: Academic, Jan. 2009, ch. 6, pp. 323–409.
[73] A. L. Yuille and A. Rangarajan, "The concave-convex procedure," *Neural Comput.*, vol. 15, no. 4, pp. 915–936, Apr. 2003.
[74] S. Niu, C. Miao, G. Chen, P. Wu, and P. Zhao, "Test-time model adaptation with only forward passes," 2024, *arXiv:2404.01650*.
[75] Q. Wang, O. Fink, L. Van Gool, and D. Dai, "Continual test-time domain adaptation," 2022, *arXiv:2203.13591*.
[76] J. Song, J. Lee, I. S. Kweon, and S. Choi, "EcoTTA: Memory-efficient continual test-time adaptation via self-distilled regularization," 2023, *arXiv:2303.01904*.

[77] Y. Hu, Y. Cheng, Z. Zhou, A. Lu, Z. Cao, and Z. Li, "FoCTTA: Low-memory continual test-time adaptation with focus," 2025, *arXiv:2502.20677*.
[78] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt, "Test-time training with self-supervision for generalization under distribution shifts," 2019, *arXiv:1909.13231*.
[79] Y. Gandelsman, Y. Sun, X. Chen, and A. A. Efros, "Test-time training with masked autoencoders," 2022, *arXiv:2209.07522*.
[80] M. Colussi, S. Mascetti, J. Dolz, and C. Desrosiers, "ReC-TTT: Contrastive feature reconstruction for test-time training," 2024, *arXiv:2411.17869*.
[81] Y. Li, N. Wang, J. Shi, X. Hou, and J. Liu, "Adaptive batch normalization for practical domain adaptation," *Pattern Recognit.*, vol. 80, pp. 109–117, Aug. 2018.
[82] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce activations, not trainable parameters for efficient on-device learning," 2020, *arXiv:2007.11622*.
[83] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "MCUNetV2: Memory-efficient patch-based inference for tiny deep learning," 2021, *arXiv:2110.15352*.
[84] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256KB memory," 2022, *arXiv:2206.15472*.
[85] R. Sebastião and J. Gama, "A study on change detection methods," in *Proc. 14th Portuguese Conf. Artif. Intell.*, 2009, pp. 353–364.
[86] A. Castellani, S. Schmitt, and B. Hammer, "Task-sensitive concept drift detector with constraint embedding," 2021, *arXiv:2108.06980*.
[87] F. V. Calderan, "An analysis of treap," Federal Univ. São Paulo (UNIFESP), São Paulo, Brazil, Tech. Rep., Jul. 2022.
[88] H. Yigit, "Time complexity of PCA 1. General time complexity of PCA," Tech. Rep., 2024. [Online]. Available: https://www.researchgate.net/publication/383360929_Time_Complexity_of_PCA_1_General_Time_Complexity_of_PCA?channel=doi&linkId=66c9c16c920e056726e495560&showFulltext=true



ASK ESPENSØNN ØREN received the Master of Science degree in cybernetics and robotics from Norwegian University of Science and Technology, in 2024. He is currently pursuing the Ph.D. degree in IoT-centric machine learning with the Department of Electrical and Photonics Engineering, Danish Technical University.



NINA KUNA PEUKER received the Bachelor of Science degree in general engineering with technological specialization in cyber systems from Danish Technical University, in 2024, where she is currently pursuing the Master of Science degree in mathematical modeling and computation with the Department of Mathematics and Computer Science.



AKIRA-MIRANDA ADEYOMI ADENIRAN-LOWE received the Bachelor of Science degree in general engineering with technological specialization in cyber systems from Danish Technical University, in 2024, where she is currently pursuing the Master of Science degree in autonomous systems with the Department of Electrical and Photonics Engineering.



SARAH RUEPP received the B.Sc. degree in electronic and computer engineering, in 2002, the M.Sc. degree in telecommunication, in 2004, and the Ph.D. degree in reliable communication networks, in 2008. She is an Associate Professor with the DTU Electro, Technical University of Denmark. She is the Head of the study of the DTU M.Sc. Program Communication Technologies and System Design. Her research in the Internet of Things focuses on leveraging innovative communication technologies to provide solutions for mission critical applications that can provide high reliability and trustworthiness to the user. One of the key aspects is the development of adaptive systems that evolve with the user. Her research interests include topics related to mission critical communication systems, network reliability and survivability, the Internet of Things, and 5G and beyond.



MARTIN NORDAL PETERSEN received the master's and Ph.D. degrees in electrical and optical communication from the Technical University of Denmark, in 2001 and 2005, respectively. Currently, he is an Associate Professor in infrastructures for Internet of Things (IoT) and applications with the Technical University of Denmark. His teaching and research areas are focused on devices and wireless communication infrastructures for low-power IoT, edge computing, embedded devices, and general IoT and applications.